

Solving Recurrences

Expansion / Iteration and Substitution Method

Goal: Convert recursive cost equations into simple growth bounds

CS332 - Design and Analysis of Algorithms

Dr. Zakir Khan

What Does “Solving a Recurrence” Mean?

Before Solving

$$T(n) = T(n - 1) + 1$$

Still depends on smaller $T(\cdot)$.

After Solving

$$T(n) = \Theta(n)$$

Now growth is expressed directly in n .

Simple Idea

A recurrence is like a cost equation that calls itself. Solving it means removing the recursive $T(\cdot)$ part and finding a direct growth class.

Session 14 Learning Targets

By the end of this session, students should be able to:

- ① Explain why a recurrence must be solved.
- ② Solve simple chain recurrences using **expansion / iteration**.
- ③ Find the correct number of expansion steps k .
- ④ Use common sums in recurrence solving.
- ⑤ Verify a guessed upper bound using the **substitution method**.
- ⑥ Avoid common mistakes such as using $k = n$ for halving recurrences.

Remember

Today is not about memorizing answers. It is about learning a repeatable solving process.

Three Methods: Which One Are We Learning?

1. Expansion

Unfold the recurrence until a pattern appears.

$$T(n) = T(n-k) + \dots$$

2. Substitution

Guess a bound, then prove it by induction.

assume $\rightarrow$ substitute

3. Tree

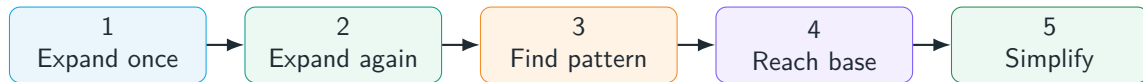
Draw recursive calls level by level. This is next session.

level cost $\rightarrow$ sum

Simple Idea

In Session 14, focus mainly on **expansion / iteration** and **substitution**. Recurrence trees come next.

Expansion / Iteration Method: 5 Easy Steps



Math Step

The most important step is choosing k , the number of times we expand until the input reaches the base case.

Example 1: Solve $T(n) = T(n-1) + 1$

Think Like a 12-Year-Old

Each recursive call reduces n by 1.

$$n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 1$$

So it is like walking down one step at a time.

Expected Answer

If we reduce by 1 each time, we need about n steps.

$$T(n) = \Theta(n)$$

Example 1: Expand Step by Step

Given:

$$T(n) = T(n-1) + 1$$

Math Step

Expand once:

$$T(n) = T(n-1) + 1$$

But:

$$T(n-1) = T(n-2) + 1$$

Substitute:

$$T(n) = T(n-2) + 1 + 1 = T(n-2) + 2$$

Example 1: Find the Pattern

Expansion	Expression
0 steps	$T(n)$
1 step	$T(n - 1) + 1$
2 steps	$T(n - 2) + 2$
3 steps	$T(n - 3) + 3$
k steps	$T(n - k) + k$

Remember

After k steps:

$$T(n) = T(n - k) + k$$

Example 1: Reach the Base Case

Base case happens when input becomes 1:

$$n - k = 1$$

Solve for k :

$$k = n - 1$$

Put this into the pattern:

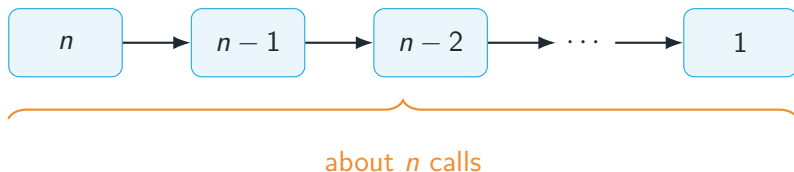
$$T(n) = T(n - k) + k$$

$$T(n) = T(1) + (n - 1)$$

Since $T(1)$ is constant:

$$T(n) = \Theta(n)$$

Visual Chain: Why the Answer is Linear



Answer

One recursive call per step, and n decreases by 1 each time. Total work is linear:

$$T(n) = \Theta(n)$$

Example 2: Solve $T(n) = T(n-1) + n$

Simple Idea

This time each level does a different amount of extra work:

$$n, n-1, n-2, \dots, 2, 1$$

So we must add many numbers.

$$T(n) = T(n-1) + n$$

Try It

Prediction: because we are adding $n + (n-1) + (n-2) + \dots + 1$, the answer should be quadratic.

Example 2: Expansion

Given:

$$T(n) = T(n-1) + n$$

Expand one step:

$$T(n-1) = T(n-2) + (n-1)$$

So:

$$T(n) = T(n-2) + (n-1) + n$$

After k steps:

$$T(n) = T(n-k) + \sum_{i=0}^{k-1} (n-i)$$

Example 2: Reach the Base Case

Base case:

$$n - k = 1 \quad \Rightarrow \quad k = n - 1$$

So the accumulated work becomes:

$$T(n) = T(1) + 2 + 3 + \cdots + n$$

Use arithmetic series:

$$2 + 3 + \cdots + n = \frac{n(n+1)}{2} - 1$$

Dominant term:

$$\frac{n^2}{2}$$

Therefore:

$$T(n) = \Theta(n^2)$$

Series Toolbox for Recurrences

Series	Formula	Growth
Constant	$1 + 1 + \dots + 1$	$\Theta(n)$
Arithmetic	$1 + 2 + \dots + n = \frac{n(n+1)}{2}$	$\Theta(n^2)$
Geometric	$1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1$	$\Theta(2^k)$

Remember

In recurrence solving, local work accumulates. Do not ignore the terms that are added at every recursive level.

Example 3: Solve $T(n) = T(n/2) + 1$

Meaning

The input is cut in half each time:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \dots \rightarrow 1$$

Real Example

This is the style of Binary Search.

halving $\Rightarrow$ logarithmic levels

Example 3: Expansion

Given:

$$T(n) = T(n/2) + 1$$

After 1 step:

$$T(n) = T(n/2) + 1$$

After 2 steps:

$$T(n) = T(n/4) + 2$$

After 3 steps:

$$T(n) = T(n/8) + 3$$

After k steps:

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

Example 3: Find k

Base case when input becomes 1:

$$\frac{n}{2^k} = 1$$

Multiply both sides by 2^k :

$$n = 2^k$$

Take $\log_2$:

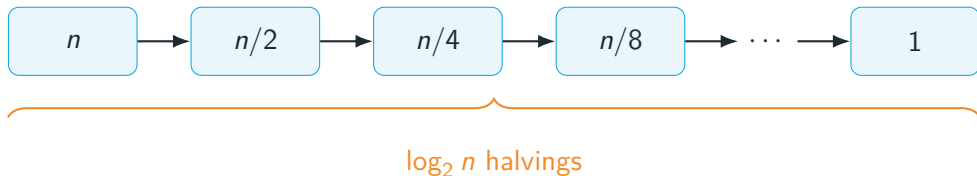
$$k = \log_2 n$$

So:

$$T(n) = T(1) + \log_2 n$$

$$T(n) = \Theta(\log n)$$

Visual: Halving Means Logarithmic



Answer

If the input halves each time, number of levels is usually $\Theta(\log n)$.

Quick Activity: How Many Halvings?

n	Halving Chain	Levels
8	$8 \rightarrow 4 \rightarrow 2 \rightarrow 1$	3
16	$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$	4
32	$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$	5
64	$64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$	6

Remember

$$k = \log_2 n$$

For $n = 64$, $k = \log_2 64 = 6$.

How to Guess the Bound Before Proof

Ask four simple questions:

- ① Does the input decrease by 1?
- ② Does the input halve?
- ③ Are there multiple recursive calls?
- ④ How much local work is added at each step?

Recurrence	Good Prediction
$T(n) = T(n - 1) + 1$	$\Theta(n)$
$T(n) = T(n - 1) + n$	$\Theta(n^2)$
$T(n) = T(n/2) + 1$	$\Theta(\log n)$
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$

Substitution Method: Guess, Assume, Verify

1. Guess

Use intuition or expansion to guess a bound.

$$T(n) \leq cn$$

2. Assume

Assume the bound for smaller input.

$$T(n-1) \leq c(n-1)$$

3. Verify

Substitute and show it fits.

$$T(n) \leq cn$$

Common Mistake

Substitution is a proof method. Expansion helps discover a pattern; substitution proves the guessed bound.

Substitution Proof 1: Verify $T(n) = O(n)$

Recurrence:

$$T(n) = T(n-1) + 1$$

Guess:

$$T(n) \leq cn$$

Induction assumption for smaller input:

$$T(n-1) \leq c(n-1)$$

Substitute into recurrence:

$$T(n) \leq c(n-1) + 1$$

$$T(n) \leq cn - c + 1$$

Substitution Proof 1: Finish

We reached:

$$T(n) \leq cn - c + 1$$

We want:

$$T(n) \leq cn$$

So we need:

$$cn - c + 1 \leq cn$$

Subtract cn from both sides:

$$-c + 1 \leq 0 \quad \Rightarrow \quad c \geq 1$$

Choose any $c \geq 1$. Therefore:

$$T(n) = O(n)$$

Substitution Proof 2: Verify $T(n) = O(\log n)$

Recurrence:

$$T(n) = T(n/2) + 1$$

Guess:

$$T(n) \leq c \log_2 n$$

Induction assumption:

$$T(n/2) \leq c \log_2(n/2)$$

Substitute:

$$T(n) \leq c \log_2(n/2) + 1$$

Use:

$$\log_2(n/2) = \log_2 n - 1$$

Substitution Proof 2: Finish

$$T(n) \leq c(\log_2 n - 1) + 1$$

$$T(n) \leq c \log_2 n - c + 1$$

To prove $T(n) \leq c \log_2 n$, we need:

$$-c + 1 \leq 0 \quad \Rightarrow \quad c \geq 1$$

Choose any $c \geq 1$. Therefore:

$$T(n) = O(\log n)$$

Common Induction-Hypothesis Mistake

For:

$$T(n) = T(n-1) + 1$$

Which assumption is correct?

Correct

$$T(n-1) \leq c(n-1)$$

We assume the bound for the smaller input.

Wrong

$$T(n) \leq cn$$

This is what we are trying to prove, not what we assume.

Bridge Example: $T(n) = 2T(n/2) + n$

$$T(n) = 2T(n/2) + n$$

Where It Appears

Merge Sort and many divide-and-conquer algorithms.

Known Result

$$T(n) = \Theta(n \log n)$$

Because there are $\log n$ levels and each level costs n .

Remember

This recurrence is easier to understand visually using recurrence trees in Session 15.

Expansion vs Substitution

Expansion / Iteration

Purpose: find the pattern.

Typical form:

$$T(n) = T(n - k) + \dots$$

Best for simple chain recurrences.

Substitution

Purpose: prove the guessed bound.

Typical flow:

assume $\rightarrow$ substitute $\rightarrow$ verify

Best for formal correctness of the bound.

Simple Idea

Simple sentence: **Expansion discovers; substitution verifies.**

Practice: Predict the Growth

Predict the tight bound for each recurrence.

Recurrence	Your Prediction
$T(n) = T(n - 1) + 1$	_____
$T(n) = T(n - 1) + n$	_____
$T(n) = T(n/2) + 1$	_____
$T(n) = 2T(n/2) + n$	_____

Try It

First predict silently. Then discuss with a partner for one minute.

Practice Answers

Recurrence	Answer	Reason
$T(n) = T(n - 1) + 1$	$\Theta(n)$	one-by-one chain
$T(n) = T(n - 1) + n$	$\Theta(n^2)$	arithmetic sum
$T(n) = T(n/2) + 1$	$\Theta(\log n)$	halving
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	$\log n$ levels, cost n each

Validation Checklist

Before writing the final answer, check:

- ① Did I state the base case?
- ② Did I expand enough times to see the pattern?
- ③ Did I write the k -th form?
- ④ Did I choose k correctly at the base case?
- ⑤ Did I simplify using the dominant term?
- ⑥ If proving with substitution, did I assume the bound for smaller input only?

Remember

Good recurrence solutions show reasoning, not only the final Θ answer.

Common Mistakes

Mistake 1

Expanding only once and guessing immediately.

Mistake 2

For $T(n/2)$, using $k = n$ instead of $k = \log n$.

Mistake 3

Ignoring accumulated local work such as $n + (n - 1) + \dots + 1$.

Common Mistake

The recursive part tells where the next call goes. The added part tells what work accumulates.

Timed Worksheet: 5 Minutes

Solve each by iteration. For each recurrence, write:

- ① after k steps,
- ② value of k at the base case,
- ③ final Θ class.

$$T(n) = T(n - 1) + 1$$

$$T(n) = T(n - 1) + n$$

$$T(n) = T(n/2) + 1$$

Try It

Work in pairs. One student writes expansion; the other checks the base-case value of k .

Worksheet Answer Key

Recurrence	After k steps	k at base	Answer
$T(n) = T(n-1) + 1$	$T(n-k) + k$	$n-1$	$\Theta(n)$
$T(n) = T(n-1) + n$	$T(n-k) + \sum_{i=0}^{k-1} (n-i)$	$n-1$	$\Theta(n^2)$
$T(n) = T(n/2) + 1$	$T(n/2^k) + k$	$\log_2 n$	$\Theta(\log n)$

Session 14 Summary

Iteration

Repeatedly expand until a pattern appears.

$$T(n - k) + \dots$$

Substitution

Guess a bound and verify it by induction.

assume $\rightarrow$ prove

Halving

Input halves give logarithmic levels.

$$k = \log_2 n$$

Expand $\rightarrow$ Find pattern $\rightarrow$ Reach base case $\rightarrow$ Simplify

Exit Ticket

Answer these before leaving:

- ① For $T(n) = T(n - 1) + 1$, why is the answer $\Theta(n)$?
- ② For $T(n) = T(n/2) + 1$, why is $k = \log_2 n$?
- ③ What is the difference between expansion and substitution?

Answer

One-line target answer: expansion helps find the pattern, while substitution proves the guessed bound.

Session 14 Complete

Expansion finds the pattern. Substitution proves the bound.

Next Session: Recurrence-Tree Method